

Nama : Mikhael Kurniawan
 Kelas : IF-A
 NPM : 5250411043
 Mata Kuliah : Struktur Data Teori

Soal Tugas

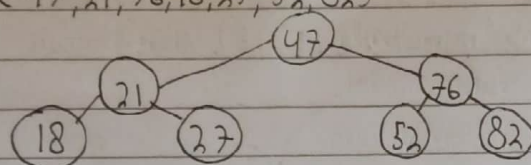
① Diberikan sekumpulan data angka sebagai berikut : $\{47, 21, 76, 18, 27, 52, 82\}$

a. Gambarkan bentuk pohon BST yg dihasilkan jika data dimasukkan sesuai urutan tersebut.

b. Jika kita ingin pohon tersebut memiliki tinggi (height) yg lebih rendah (lebih seimbang) dari gambar pada poin (a), tentukan urutan input data yg baru dari 7 angka tersebut yg dpt menghasilkan pohon yg lebih efisien. Gambarkan pohon barunya dan jelaskan secara logika mengapa urutan input sangat memengaruhi efisiensi pencarian pada BST.

Jawab :

a. BST dari : $\{47, 21, 76, 18, 27, 52, 82\}$



47 = Root

• $18 < 47 \rightarrow$ masuk kiri-kiri

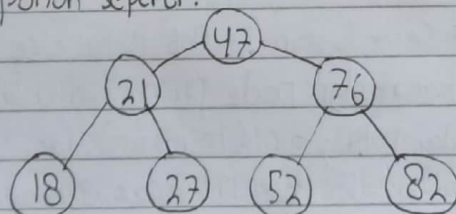
• $52 > 47 \rightarrow$ masuk kanan-kiri

$21 < 47 \rightarrow$ masuk kiri • $27 < 47 \rightarrow$ masuk kiri-kanan

• $82 > 47 \rightarrow$ masuk kanan-kanan

$76 > 47 \rightarrow$ masuk kanan

b. Dari kode data angka sudah memiliki tinggi (height) yg cukup seimbang dgn 47 sebagai ~~21~~ median. Namun kita bisa mencobanya dgn urutan : $\{47, 76, 21, 52, 82, 18, 27\}$, yg akan membentuk pohon seperti :



47 = root

• $52 > 47 \rightarrow$ masuk kanan-kiri

• $18 < 47 \rightarrow$ masuk kiri-kiri

$76 > 47 \rightarrow$ masuk kanan

• $82 > 47 \rightarrow$ masuk kanan-kanan

• $27 < 47 \rightarrow$ masuk kiri-kanan

$21 < 47 \rightarrow$ masuk kiri

• Urutan input sangat memengaruhi efisiensi pencarian pada BST karena node pertama yg dimasukkan akan menjadi root dan setiap node baru berikutnya akan ditempatkan berdasarkan perbandingan dari root ke bawah. Dengan memasukkan node median (47), maka data akan terbagi rata dgn tinggi (height) = 3 dan pencarian hanya membutuhkan

maksimal 3 perbandingan ($O(\log n)$)

2. Diberikan Fungsi `min_value_node` pada sebuah kode. Fungsi tersebut hanya mencari nilai paling minimum (Paling kiri). Buatlah Fungsi baru dgn nama `def Find_second_min()` yg mana Fungsi baru ini bertujuan untuk mencari nilai terkecil kedua dlm pohon (tuliskan hanya kode program dari Fungsi `def Find_second_min()` pada lembar jawaban Anda). Jelaskan langkah logika Anda terutama jika node paling kiri memiliki anak/cabang disebelah kanan? Dan bagaimana jika pohon hanya memiliki dua node (root dan 1 node anak saja)?

Jawab:

```
def Find_second_min(self):
```

```
    if self.root is None: } Jika root kosong  
        return None
```

```
    if self.root.left is None and self.root.right is None: } Jika hanya ada root  
        return None
```

```
    min_node = self.min_value_node(self.root)
```

```
    if min_node.right is not None: } node minimum punya anak  
        return self.min_value_node(min_node.right).value } kanan
```

```
    second_min = None
```

```
    temp = self.root
```

```
    while temp is not None:
```

```
        if temp.value > min_node.value:
```

```
            second_min = temp
```

```
            temp = temp.left
```

```
        elif temp.value < min_node.value:
```

```
            temp = temp.right
```

```
        else:
```

```
            break
```

```
    return second_min.value if second_min is not None else None
```

- Jika node paling kiri memiliki anak kanan dgn node paling kiri adalah node minimum, maka Node terkecil kedua adalah nilai paling kiri (minimum) dari sub-pohon kanan node minimum itu. Contoh: node minimum = 18, memiliki anak 19, maka `second_min = 19`
- Namun jika pohon hanya memiliki dua node (root + 1 anak), maka jika anaknya di kiri `second_min` adalah root, sedangkan jika anaknya di kanan, `second_min` adalah anak kanan tersebut.

3. Diberikan Sebuah Graph dgn vertex: $\{A, B, C, D, E\}$. Dari kondisi graph awal tersebut, lakukan urutan operasi berikut secara berurutan:

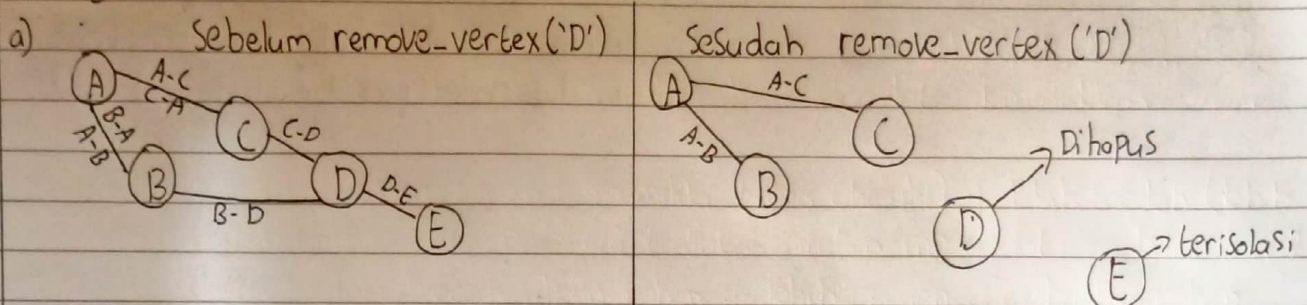
Langkah ke-1: `add-edge('A','B')`, `add-edge('A','C')`, `add-edge('B','D')`, `add-edge('C','D')`, `add-edge('D','E')`.

Langkah ke-2: `remove_vertex('D')`.

Berdasarkan langkah-langkah yg dilakukan :

- Gambarkan representasi visual Graph tersebut sebelum dan sesudah Vertex 'D' dihapus!
- Tuliskan isi dari `adj-list` secara detail untuk setiap vertex yg tersisa setelah penghapusan 'D'. Jelaskan apa yg terjadi pada edge yg sebelumnya terhubung ke 'D'!

Jawab:



b) Isi dari `adj-list` setelah `remove_vertex('D')`:

Vertex	adj-list	Pada edge yg sebelumnya terhubung ke 'D' akan ikut menghilang setelah <code>remove_vertex('D')</code> . Selain menghapus key 'D' dari <code>adj-list</code> , Fungsi juga akan menelusuri semua vertex lain dan menghapus 'D' dari daftar mereka. Vertex B dan C kehilangan adjacency-nya. Vertex E menjadi isolated vertex dgn <code>adj-list</code> kosong.
'A'	['B', 'C']	
'B'	['A']	
'C'	['A']	
'E'	[]	

4. Dalam teori graph, dikenal istilah "Degree" (derajat) yaitu jumlah edge yg terhubung pada suatu vertex. Pada class `Graph` yg sudah kita pelajari, belum ada Fungsi baru untuk menghitung ini. Tuliskan sebuah Fungsi baru (hanya tulis kode program Fungsi baru ini saja menggunakan bahasa `python`) dgn nama `get_degree(self, vertex)` yg menerima input nama vertex dan mengembalikan jumlah tetangganya berdasarkan data di `self.adj-list`. Tambahkan logika pengecekan untuk edge case jika vertex yg dicari tidak ada di dlm graph!

Jawab:

```
def get_degree(self, vertex):
    if vertex not in self.adj-list:
        print(f"vertex '{vertex}' tidak ditemukan dalam graph.")
        return None
    return len(self.adj-list[vertex])
```

Vertex Gdtkada dlm graph

Penjelasan:

`len(self.adj-list[vertex])` akan menghitung jumlah edge yg terhubung ke vertex dan Pengecekan jika vertex yg dicari tdk ada di dlm graph adapada kode `if vertex not in self.adj-list`.